

Warum diese Entscheidungen in einem ML-Projekt?

Kompakte Projektvorbereitung für klassische Textklassifikation, Evaluation und Retraining

Ziel des Dokuments: Nicht nur scikit-learn-Code ausführen, sondern erklären können, warum eine Methode gewählt wird, wie ihre Qualität bewertet wird und welche Konsequenzen Fehler im Modell haben.

1. Warum verwende ich hier TF-IDF?

Weil die Eingaben Texte sind und ein klassischer Klassifikator numerische Merkmale benötigt.

TF-IDF wandelt Wörter bzw. Wortgruppen in numerische Vektoren um. Häufige, wenig unterscheidende Wörter erhalten weniger Gewicht; charakteristische Wörter erhalten relativ mehr Gewicht.

- Gut geeignet als einfache, schnelle Baseline für Textklassifikation.
- Mit `ngram_range=(1, 2)` können zusätzlich Wortpaare berücksichtigt werden.
- Das Ergebnis ist eine meist sehr große, aber dünn besetzte Feature-Matrix (sparse matrix).

Projektbezug: TF-IDF ist besonders nützlich, wenn nachvollziehbare klassische Textmodelle gegen komplexere Ansätze oder LLMs verglichen werden sollen.

2. Warum Logistic Regression statt K-Means?

Weil hier eine überwachte Klassifikation vorliegt: Für Trainingsbeispiele sind Zielklassen bereits bekannt.

- K-Means ist unüberwachtes Lernen: Es kennt keine Zielklassen und gruppiert Daten nur nach Ähnlichkeit.
- Logistic Regression ist überwachtes Lernen: Sie lernt die Beziehung zwischen Features und bekannten Labels.
- Für binäre Klassifikation kann Logistic Regression zusätzlich Wahrscheinlichkeiten über `predict_proba()` liefern.

K-Means: $X \rightarrow \text{Cluster}$

Logistic Regression: $X + y \rightarrow \text{bekannte Klasse vorhersagen}$

3. Warum brauche ich Ground Truth?

Ground Truth ist die fachlich als richtig festgelegte Antwort. Bei überwachten Modellen sind dies die Labels, gegen die Vorhersagen verglichen werden.

- Vorhersagen objektiv vergleichen
- Precision, Recall, F1 und Accuracy berechnen
- False Positives und False Negatives identifizieren
- Retraining und Modellversionen nachvollziehbar vergleichen

Wichtig: Wenn die Ground Truth fehlerhaft oder uneindeutig ist, kann auch die Modellbewertung irreführend sein. Labelqualität ist daher ein Teil der Modellqualität.

4. Was sagt mir ein Recall von 0,70?

Recall beantwortet die Frage: Von allen tatsächlich positiven Fällen - wie viele wurden gefunden?

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

Ein Recall von 0,70 bedeutet: 70 % der tatsächlich positiven Fälle wurden erkannt; 30 % wurden als False Negatives übersehen.

Projektbezug: Bei sicherheits- oder risikorelevanten Klassifikationsaufgaben kann Recall besonders wichtig sein, weil übersehene positive Fälle fachlich teuer sein können. Welche Metrik priorisiert wird, ist aber eine fachliche Entscheidung.

5. Was passiert, wenn ich den Threshold absenke?

Der Threshold ist die Entscheidungsgrenze, ab der eine positive Klasse vergeben wird.

Confidence = 0,42
Threshold 0,50 -> nicht_relevant
Threshold 0,30 -> relevant

- Niedrigerer Threshold: mehr positive Vorhersagen, häufig höherer Recall, aber potenziell mehr False Positives.
- Höherer Threshold: weniger positive Vorhersagen, häufig höhere Precision, aber potenziell mehr False Negatives.
- Das Modell wird dabei nicht neu trainiert; nur die Entscheidungsregel ändert sich.

6. Warum kann ein F1 von 1,0 bei einem einzelnen Split irreführend sein?

Ein einzelner Train/Test-Split ist nur eine mögliche Aufteilung des Datensatzes. Besonders bei kleinen Datensätzen kann ein zufällig günstiger Testsplitt ein perfektes Ergebnis erzeugen, obwohl das Modell auf anderen Daten deutlich schlechter abschneidet.

- Kleine Testmengen reagieren stark auf einzelne Beispiele.
- Ähnliche Formulierungen können zufällig in Training und Test besonders günstig verteilt sein.
- Ein einzelner Wert sagt wenig über Stabilität aus.

7. Warum Cross-Validation?

Cross-Validation bewertet dasselbe Modell über mehrere systematische Datenaufteilungen. Bei 5-Fold-Cross-Validation dient jeder Fold einmal als Testmenge, während die übrigen vier Folds zum Training verwendet werden.

- Weniger abhängig von einem einzelnen zufälligen Split
- Bessere Schätzung der Generalisierungsleistung
- Schwankungen zwischen den Folds werden sichtbar
- Mittelwert und Standardabweichung liefern gemeinsam eine Aussage über Leistung und Stabilität

Beispiel: $F1 = 0,67 \pm 0,20$

-> mittlere Leistung 0,67, aber deutliche Schwankungen zwischen den Folds

8. Was mache ich mit False Negatives?

False Negatives sind tatsächlich positive Fälle, die vom Modell negativ klassifiziert wurden. Error Analysis untersucht diese Fälle einzeln und sucht nach systematischen Ursachen.

- Konkrete FN-Texte lesen und Muster sammeln
- Prüfen, ob ähnliche Formulierungen in den Trainingsdaten fehlen
- Ground-Truth-Label auf Plausibilität prüfen
- Threshold überprüfen
- Feature-Repräsentation prüfen, z. B. n-Gramme oder Embeddings
- Neue, fachlich bestätigte Beispiele gezielt in den Trainingsbestand aufnehmen

9. Wann retrainiere ich ein Modell?

Retraining ist sinnvoll, wenn neue verlässliche Trainingsdaten vorliegen oder sich Daten, Sprache, Kategorien oder fachliche Anforderungen verändern.

- Neue annotierte Daten stehen zur Verfügung
- Labels wurden korrigiert
- Wiederkehrende Fehlermuster wurden gefunden
- Datenverteilung oder Sprache hat sich verändert (Data Drift / Concept Drift)
- Neue Kategorien oder Formulierungen treten auf
- Eine neue Modellversion soll gegen die bisherige Version getestet werden

Wichtig: Retraining ist nicht automatisch eine Verbesserung. Modell V2 sollte auf einer vergleichbaren, unabhängigen Evaluation gegen Modell V1 geprüft werden.

10. Was ist Class Imbalance und warum ist das wichtig?

Class Imbalance bedeutet, dass die Klassen stark unterschiedlich häufig vorkommen, z. B. 95 % nicht_relevant und 5 % relevant.

```
95 % nicht_relevant
5 % relevant
```

Ein triviales Modell, das immer nicht_relevant vorhersagt, hätte dann 95 % Accuracy - wäre aber für die positive Klasse völlig unbrauchbar.

- Accuracy allein kann bei unausgeglichene Klassen irreführend sein.
- Recall, Precision und F1 der seltenen Klasse werden wichtiger.
- Stratified Splits helfen, die Klassenverteilung in Train/Test ähnlich zu halten.
- Bei geeigneten Modellen kann `class_weight="balanced"` getestet werden.

11. Wie vergleiche ich Modelle sauber?

Modelle sollten auf denselben Daten, mit derselben Vorverarbeitung und derselben Bewertungslogik verglichen werden.

TF-IDF -> Logistic Regression
TF-IDF -> Linear SVC
TF-IDF -> Naive Bayes

- Gleiche Cross-Validation-Folds verwenden
- Gleiche Zielmetrik verwenden
- Nicht nur den besten Einzelwert betrachten, sondern Mittelwert und Streuung
- Laufzeit, Erklärbarkeit und Betriebsaufwand ebenfalls berücksichtigen

Baseline-Prinzip: Zuerst ein einfaches Modell etablieren. Ein komplexeres Modell lohnt sich nur, wenn es einen relevanten Mehrwert liefert.

12. Was ist Data Leakage?

Data Leakage entsteht, wenn Informationen in das Training gelangen, die zum Vorhersagezeitpunkt eigentlich nicht verfügbar wären oder die Zielvariable indirekt verraten.

- Beispiel: Für die Vorhersage einer SLA-Verletzung wird die tatsächliche spätere Bearbeitungsdauer als Feature verwendet.
- Das Modell wirkt dann im Test sehr gut, lernt aber eine Information, die in der realen Anwendung noch gar nicht bekannt wäre.
- Preprocessing sollte innerhalb einer Pipeline auf Trainingsdaten gelernt werden, damit keine Testinformationen einfließen.

Prüffrage: Ist jedes Feature zum echten Vorhersagezeitpunkt verfügbar? Wenn nein, besteht Leakage-Risiko.

13. Train, Validation und Test - wofür brauche ich welche Daten?

Für kleine Lernbeispiele reicht häufig Train/Test plus Cross-Validation. In größeren Projekten werden die Rollen stärker getrennt.

Datenteil	Zweck
Training	Modellparameter lernen
Validation / Cross-Validation	Modellvarianten und Hyperparameter vergleichen
Test	Abschließende, möglichst unverfälschte Bewertung

Wichtig: Das Testset sollte nicht wiederholt zur Auswahl von Modell oder Hyperparametern benutzt werden, sonst optimiert man indirekt auf den Test.

14. Was ist Hyperparameter-Tuning?

Hyperparameter werden nicht direkt aus den Daten gelernt, sondern vor dem Training festgelegt, z. B. C bei Logistic Regression oder ngram_range bei TF-IDF.

Beispiele:
C = [0.1, 1, 10]
ngram_range = [(1,1), (1,2)]

Werkzeuge wie GridSearchCV testen Kombinationen innerhalb einer Cross-Validation. Entscheidend ist, dass die Auswahl nicht anhand des finalen Testsets erfolgt.

15. Ist predict_proba() automatisch eine echte Wahrscheinlichkeit?

predict_proba() liefert modellbasierte Wahrscheinlichkeitswerte. Diese sind nicht automatisch perfekt kalibriert. Eine Ausgabe von 0,80 bedeutet nicht zwangsläufig, dass in der Realität exakt 80 % solcher Fälle positiv sind.

- Für Threshold-Entscheidungen sind Wahrscheinlichkeiten sehr nützlich.
- Bei produktiven Systemen kann zusätzlich geprüft werden, ob die Wahrscheinlichkeiten kalibriert sind.
- Confidence sollte nicht mit absoluter Sicherheit gleichgesetzt werden.

16. Wann reichen TF-IDF-Features, wann sind Embeddings interessant?

TF-IDF bewertet hauptsächlich Wörter und Wortgruppen. Embeddings repräsentieren Texte als dichte Vektoren, die semantische Ähnlichkeit besser erfassen können.

Ansatz	Stärken	Typischer Fokus
TF-IDF	Einfach, schnell, transparent, starke Baseline	Wortlaut und n-Gramme
Embeddings	Semantische Ähnlichkeit, Clustering, moderne Features	Bedeutungsähnlichkeit auch bei anderem Wortlaut

17. Warum sind Reproduzierbarkeit und Modellversionen wichtig?

In einem Projekt sollte nachvollziehbar sein, mit welchen Daten, Parametern und Code-Versionen ein Modell erzeugt wurde.

- random_state dokumentieren, wenn Zufall beteiligt ist
- Datensatz- und Label-Version dokumentieren
- Modellparameter und Preprocessing gemeinsam versionieren
- Metriken pro Modellversion speichern
- Nur vergleichbare Evaluationen gegenüberstellen

18. Was passiert nach dem Training in Produktion?

Eine gute Offline-Evaluation ist nur der Start. In Produktion muss beobachtet werden, ob sich Eingaben, Fehlerarten oder Klassenverteilungen verändern.

- Verteilung der Vorhersagen beobachten
- False Positives und False Negatives aus fachlicher Rückmeldung sammeln
- Drift erkennen
- Threshold oder Modell nur kontrolliert ändern
- Neue Ground Truth für spätere Retrainings aufbauen

19. Generisches Grundschema eines ML-Projekts

Daten erfassen

-> Daten prüfen / bereinigen

-> X und y definieren

-> Train / Validation / Test festlegen

- > Preprocessing
- > Baseline-Modell trainieren
- > predict / predict_proba
- > Precision / Recall / F1 / Confusion Matrix
- > Error Analysis
- > Modellvergleich / Tuning
- > finaler Test
- > Deployment
- > Monitoring
- > neue Ground Truth
- > Retraining

20. Fragen, die ich im Projekt erklären können sollte

- Warum ist dieses Problem Klassifikation und nicht Clustering?
- Woran erkenne ich, ob ich supervised oder unsupervised learning brauche?
- Welche Ground Truth steht zur Verfügung und wie zuverlässig sind die Labels?
- Wer erzeugt bzw. bestätigt die Ground Truth fachlich?
- Welche Fehlerart ist fachlich kritischer: False Positive oder False Negative?
- Welche Metrik passt deshalb zur Aufgabe?
- Warum kann Accuracy bei unausgeglichenen Klassen irreführend sein?
- Wann ist Recall wichtiger als Precision - und wann umgekehrt?
- Was sagt mir die Confusion Matrix zusätzlich zu F1?
- Wie wurde verhindert, dass Data Leakage entsteht?
- Sind alle verwendeten Features zum echten Vorhersagezeitpunkt verfügbar?
- Warum wird Preprocessing in einer Pipeline mit dem Modell verbunden?
- Warum verwende ich stratify=y beim Train/Test-Split?
- Was bewirkt random_state und warum dokumentiert man ihn?
- Wie stabil ist das Ergebnis über Cross-Validation-Folds?
- Was bedeutet eine hohe Standardabweichung der Cross-Validation-Scores?
- Ist die Datenmenge ausreichend und sind die Klassen unausgeglichen?
- Wie erkenne ich Overfitting?
- Warum wurde dieses Modell gegenüber einer einfacheren Baseline gewählt?
- Wie vergleiche ich Logistic Regression, Linear SVC und Naive Bayes fair?
- Wie wurde der Threshold bestimmt?
- Was passiert mit Precision und Recall, wenn ich den Threshold ändere?
- Sind die predict_proba-Werte ausreichend kalibriert oder nur als Confidence zu verstehen?
- Wann reichen TF-IDF-Features und wann wären Embeddings sinnvoll?
- Wann wäre ein klassisches ML-Modell sinnvoller als ein LLM?
- Welche Fehlerfälle treten wiederholt auf und was lerne ich daraus?
- Wie werden False Positives und False Negatives systematisch untersucht?
- Wie werden neue Ground-Truth-Daten gesammelt und für Retraining verwendet?
- Wann ist Retraining wirklich notwendig und wann reicht eine Threshold-Anpassung?
- Wie wird Modell V2 fair gegen Modell V1 verglichen?
- Welche Daten-, Code- und Modellversion gehört zu einem bestimmten Ergebnis?
- Wie wird nach Deployment überwacht, ob das Modell schlechter wird?
- Wie erkenne ich Data Drift oder veränderte Formulierungen in neuen Daten?

- Welche Metriken und Fehlertypen sollten im Betrieb regelmäßig überwacht werden?

21. Merksatz

Ein gutes ML-System ist nicht einfach das Modell mit dem höchsten Score. Entscheidend sind verlässliche Daten und Labels, passende Metriken, stabile Validierung, nachvollziehbare Fehleranalyse, saubere Modellvergleiche und kontrolliertes Retraining.